



Programa Certified AI-LLM Integration Engineer

Curso de Fundamentos de LLMs y APIs

www.bsginstitute.com

Andrés Felipe Rojas Parra

- Ingeniero electrónico con mas de 23 años de experiencia
- PGP Artificial Intelligence & Machine Learning at Texas University – Austin, TX
- Master en Inteligencia Artificial y Big Data – IEBSchool
- Estudiante de PGP en Cloud Computing at Texas University - Austin, TX
- Texas Executive Education Program – McCombs School of Business – Decision Science and AI Immersion Program
- Especialización en MLOps de la Universidad de Duke



arojaspa@outlook.com



@arojaspa



<https://www.linkedin.com/in/arojaspa/>



+573005906373



Capítulo 1 — Bases Arquitectónicas

Sesión 2

Implementación LLM: Local, Cloud y Escalabilidad

1

FastAPI + Ollama
Windows Local

2

Azure AI Foundry
Detección Fraude

3

Patrones y
Escalabilidad

Agenda de la Sesión 2

0

1

FastAPI + Ollama — Windows Local

~60 min

- Instalación Python 3.12 + Ollama en Windows
- API de análisis de riesgo crediticio
- LLM 100% local — sin costo por token
- Demo: `/credit/analyze` con Llama 3.2:3b

0

2

Azure AI Foundry — Fraude en la Nube

~60 min

- Configurar proyecto en Azure AI Foundry
- Autenticación: API Key vs Managed Identity
- API de detección de fraude con GPT-4o
- Demo: `/fraud/analyze` con transacción sospechosa

0

3

Patrones y Escalabilidad LLM

~60 min

- Microservicios y contenedores Docker
- Escalabilidad horizontal y vertical
- Integración con APIs y mensajería
- Kubernetes + HPA para LLMs

T E M A 1

Análisis de Riesgo Crediticio — Local

FastAPI + Ollama (Llama 3.2:3b) en Windows — 100% privado, \$0 en tokens

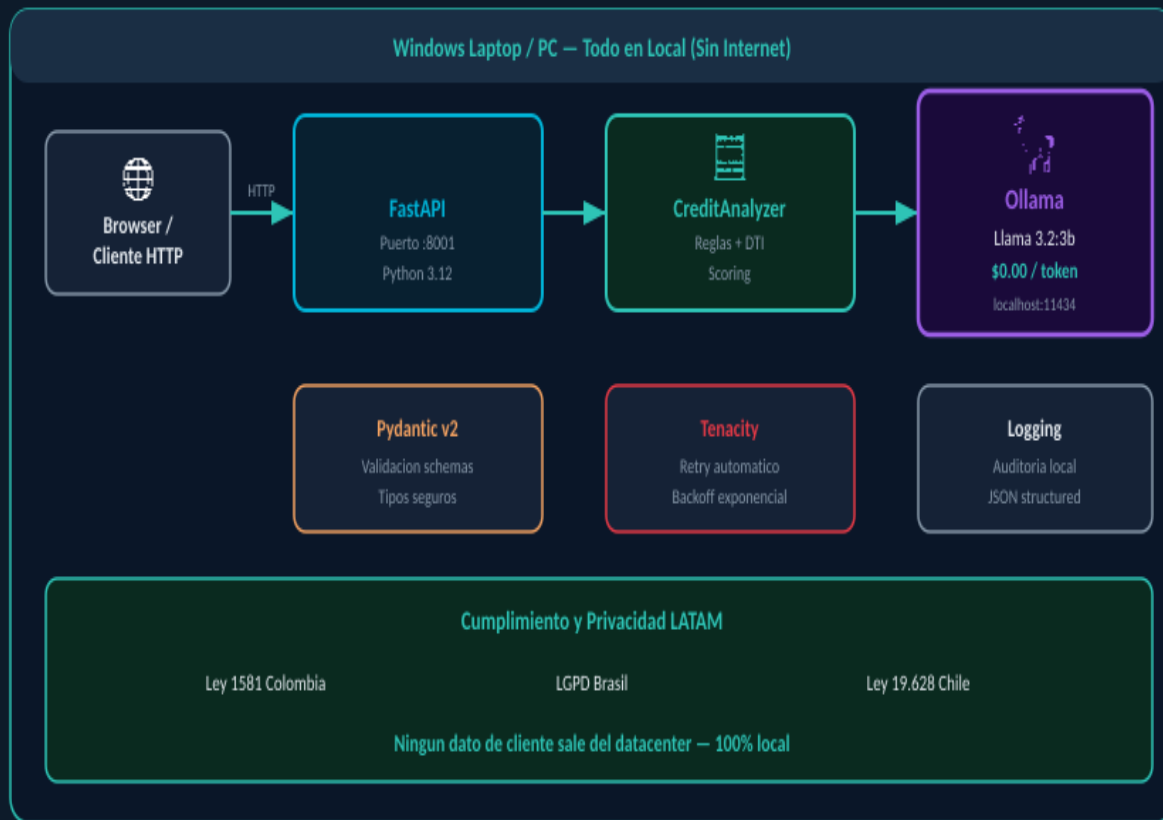
Arquitectura Local — FastAPI + Ollama en Windows

Instalación Windows

1. `winget install Python.Python.3.12`
2. `winget install Ollama.Ollama`
3. `ollama pull llama3.2:3b`
4. `pip install -r requirements.txt`
5. `make run-local`

Endpoints API

- POST `/credit/analyze`
- POST `/credit/score`
- GET `/credit/demo`
- GET `/health`



Instalación Paso a Paso — Windows (Python 3.12 + Ollama)

1

Instalar Python 3.12

```
winget install Python.Python.3.12  
python --version # Python 3.12.x
```

2

Instalar Ollama

```
winget install Ollama.Ollama  
# 0 descargar desde  
https://ollama.ai/download/windows
```

3

Descargar modelo LLM

```
ollama pull llama3.2:3b  
# Descarga ~2GB – esperar completar
```

4

Configurar proyecto

```
python -m venv .venv  
.venv\Scripts\activate  
pip install -r requirements.txt  
copy .env.example .env
```

5

Ejecutar API

```
make run-local  
# 0: uvicorn examples.local_analyzer.main:app  
--port 8001 --reload  
# Abrir: http://localhost:8001/docs
```

Demo — Análisis Crediticio con Ollama (código clave)

analyzer.py — Cálculo DTI + LLM

```
def analyze(self, req: CreditRequest):
    indicators = calculate_indicators(req)
    score = calculate_risk_score(req, indicators)

    # LLM local via Ollama
    prompt = self._build_prompt(req, indicators, score)
    llm_response = self._call_llm(prompt)
    parsed = self._parse_llm_response(llm_response)

    return CreditAnalysisResponse(
        risk_score=score,
        recommendation=parsed["recommendation"],
        ai_analysis=parsed["analysis"],
        model_used=self.model # "llama3.2:3b"
    )
```

main.py — Endpoint FastAPI

```
@app.post("/credit/analyze",
          response_model=CreditAnalysisResponse)
async def analyze_credit(
    request: CreditRequest,
    background_tasks: BackgroundTasks
):
    result = analyzer.analyze(request)
    background_tasks.add_task(
        _log_analysis, result.request_id
    )
    return result
```

curl — Test del endpoint

```
curl -X POST http://localhost:8001/credit/analyze \
-H "Content-Type: application/json" \
-d '{"applicant": {...},
    "credit_type": "personal",
```

Respuesta JSON:

```
{"risk_score": 74, "risk_level": "bajo", "recommendation": "APROBAR", "indicators": {"debt_to_income_ratio": 28.4,
"estimated_monthly_payment_usd": 283.50}, "ai_analysis": "El solicitante presenta un DTI de 28.4% dentro del umbral
regulatorio SFC...", "privacy_note": "Análisis procesado 100% local"}
```


T E M A 2

Detección de Fraude en Azure AI Foundry

GPT-4o en Azure | SLA 99.9% | Regiones LATAM | Responsable AI integrado

Azure AI Foundry — Arquitectura de Detección de Fraude

Azure AI Foundry

Plataforma Microsoft para desplegar modelos de IA enterprise (GPT-4o, Phi-4, Llama).

Configuracion .env

AZURE_AI_ENDPOINT=

<https://<proj>.services.ai.azure.com/models>

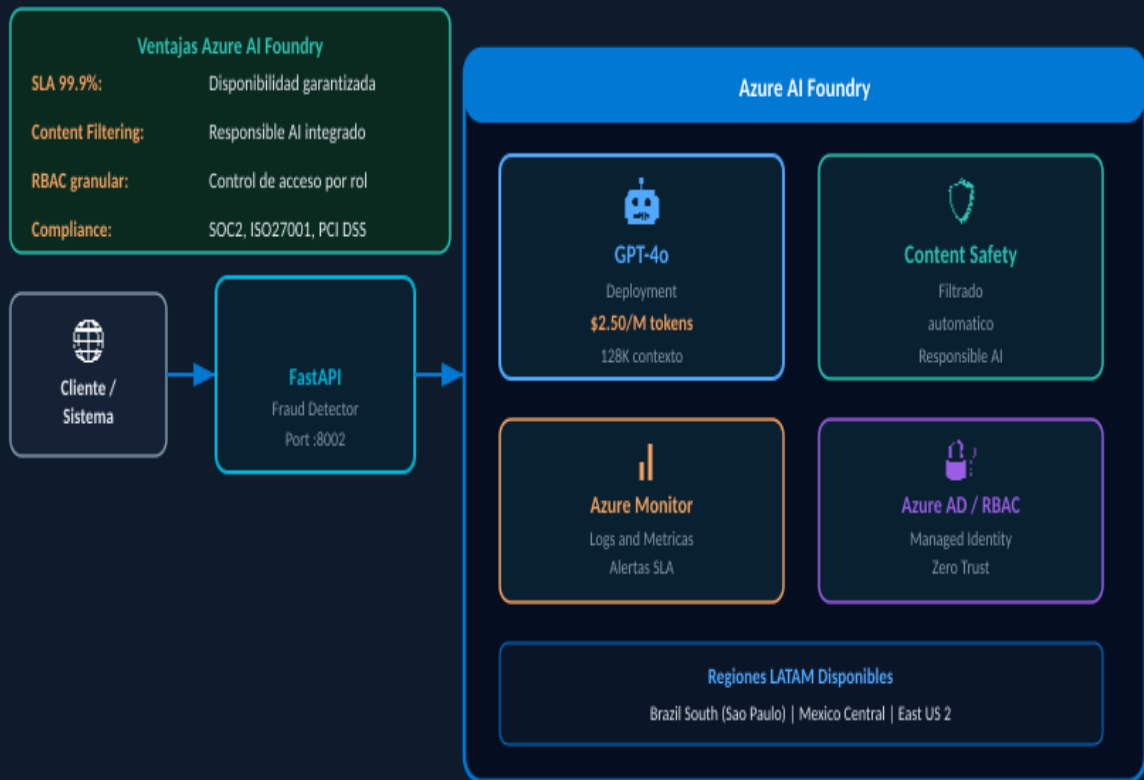
AZURE_AI_KEY=<key>

AZURE_AI_MODEL=gpt-4o

Autenticacion

Dev: API Key (sencillo)

Prod: Managed Identity (seguro, sin secretos en codigo)



Demo — Detección de Fraude con Azure AI Foundry (código clave)

fraud_detector.py — Cliente Azure OpenAI SDK

```
# Azure AI Foundry usa el SDK compatible con OpenAI
from openai import AzureOpenAI

self._client = AzureOpenAI(
    azure_endpoint=AZURE_ENDPOINT,
    api_key=AZURE_KEY,
    api_version="2024-12-01-preview"
)

# Llamada con retry automatico (tenacity)
@retry(stop=stop_after_attempt(3),
        wait=wait_exponential(min=2, max=10))
def _call_azure(self, prompt: str) -> str:
    response = self._client.chat.completions.create(
        model=self.model,      # "gpt-4o"
        messages=[...],
        temperature=0.1,      # Bajo para auditoria
        max_tokens=500
    )
    return response.choices[0].message.content
```

def _calculate_heuristics(req): Señales de fraude detectadas

```
# Monto vs. promedio del usuario
ratio = req.amount / req.user_profile.avg_tx
if ratio > 10:
    signals.append(f"Monto {ratio}x el promedio")

# Pais inusual
if req.merchant.country not in
req.user_profile.typical_countries:
    signals.append("Pais inusual")

# Alta frecuencia
if req.user_profile.transactions_24h > 10:
    signals.append("10+ tx en 24h")
```

Test: transaccion sospechosa

```
curl http://localhost:8002/fraud/demo
# Respuesta:
# "fraud_verdict": "BLOQUEAR"
# "fraud_probability": 0.89
# "explanation": "Transaccion 57x superior
# al promedio, pais inusual (US), hora
# nocturna. Patron tipico de fraude CNP."
```

TEMA 3

Patrones Arquitectónicos y Escalabilidad LLM

Microservicios | Docker | Kubernetes | HPA | Mensajería | Zero-downtime

Escalabilidad Horizontal, Vertical, Microservicios y Kubernetes

Horizontal (Scale-Out)

Mas pods, mismo hardware. Ideal LLM stateless. HPA automatico con K8s.

Vertical (Scale-Up)

Mejor hardware / GPU. Llama 3.1 70B necesita 40GB+ VRAM. A100/H100.

Microservicios

Cada componente escala independiente. Gateway, Auth, LLM separados.

Mensajeria

RabbitMQ / Kafka desacoplan carga. El LLM consume a su ritmo.

Escalamiento Horizontal (Scale-Out)



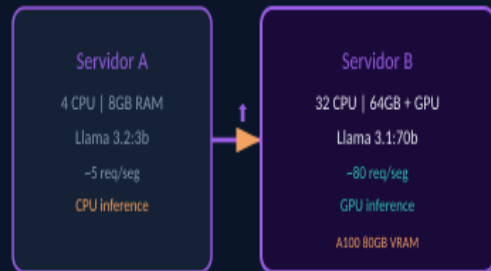
+ HPA agrega pods automaticamente cuando CPU > 70%

minReplicas: 2 | maxReplicas: 10

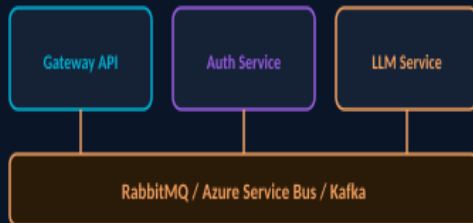
Ideal LLM: stateless, facil de replicar

Costo: lineal con carga real

Escalamiento Vertical (Scale-Up)

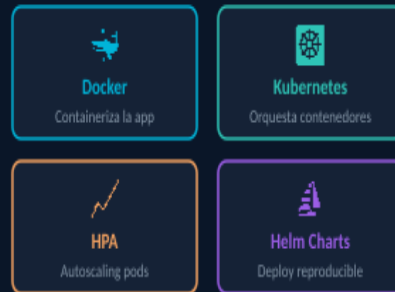


Microservicios + Bus de Mensajeria



Desacoplamiento: cada servicio escala de forma independiente

Contenedores y Orquestacion



kubectl apply -f k8s/ | helm upgrade --install llm-api ./charts

Docker Compose y Kubernetes — Infraestructura completa como código

docker-compose.yml — Stack completo local

```
services:
  api-local:
    build: { context: ., dockerfile: docker/Dockerfile.local }
    ports: ["8001:8001"]
    depends_on: { ollama: { condition: service_healthy } }

  ollama:
    image: ollama/ollama:latest
    ports: ["11434:11434"]
    volumes: [ollama_data:/root/.ollama]
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:11434/api/tags"]

  redis:
    image: redis:7-alpine # Cache de respuestas

  grafana:
    image: grafana/grafana:latest
    ports: ["3000:3000"]
```

k8s/deployment.yaml — HPA en Kubernetes

```
# HorizontalPodAutoscaler
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
spec:
  scaleTargetRef:
    kind: Deployment
    name: llm-api
  minReplicas: 2
  maxReplicas: 10 # Hasta 10 pods
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          averageUtilization: 70 # CPU>70% -> scale
    - type: Resource
      resource:
        name: memory
        target:
          averageUtilization: 80
```

Comandos operacion diaria

```
make docker-up      # Levantar stack completo
make k8s-deploy     # Desplegar en Kubernetes
kubectl get hpa      # Ver autoscaling
kubectl top pods     # Ver consumo recursos
make logs            # Ver logs en tiempo real
```

Cuándo usar Local (Ollama) vs Azure AI Foundry — Decisión arquitectónica

La elección depende de: privacidad de datos, presupuesto, SLA requerido y volumen de peticiones.

Criterio	 Local (Ollama)	 Azure AI Foundry
Privacidad datos	✓ 100% local — cero riesgo	⚠ Datos al proveedor (DPA requerido)
Costo por token	✓ \$0 — solo electricidad	💰 \$0.15–\$10 por 1M tokens
Calidad del modelo	⚠ Llama 3.2:3b (bueno)	✓ GPT-4o (mejor calidad)
Latencia	⚠ 3-8s (sin GPU)	✓ 0.8-2s (GPT-4o API)
Escalabilidad	⚠ Limitado por hardware	✓ Escalado automático
SLA garantizado	✗ Depende de tu infra	✓ 99.9% Microsoft SLA
Compliance LATAM	✓ Datos en tus servidores	✓ Regiones Brasil/Mexico
Mantenimiento	⚠ Tú actualizas modelos	✓ Managed por Microsoft
Caso ideal	Bancos, salud, gobierno	Startups, apps públicas

 Patrón híbrido LATAM: datos sensibles → Local (Ollama) | datos no-PII, alto volumen → Azure AI Foundry

Proyecto Final Sesión 2 — Entregables y Criterios de Evaluación

Entregables del Proyecto

1. Repositorio GitHub estructurado
 - README con instrucciones Windows
 - requirements.txt y Makefile funcional
2. Tema 1 — API Local funcionando
 - Al menos 2 endpoints documentados
 - Demo con Ollama ejecutando en local
 - Test con curl o Swagger UI
3. Tema 2 — API Azure (o mock)
 - Cliente Azure AI Foundry configurado
 - Endpoint /fraud/analyze documentado
 - Manejo de errores y retry
4. Tema 3 — Infraestructura
 - docker-compose.yml funcional
 - k8s/deployment.yaml con HPA
 - README de despliegue

Criterios de Evaluación (100 pts)

API Local funcional — 30 pts

API Azure / Mock — 25 pts

Docker + K8s — 20 pts

Documentación — 15 pts

 Fecha de entrega: Antes de la Sesión 3
 Entregar: PR al repositorio del curso

Resumen y Tarea para esta Semana

✓ Aprendimos Hoy

- FastAPI + Ollama 100% local en Windows — sin costo por token
- Azure AI Foundry con GPT-4o — SLA 99.9% y regiones LATAM
- Docker Compose para stack completo de desarrollo local
- Kubernetes + HPA: escalado automático de 2 a 10 pods
- Local vs Azure: decisión arquitectónica basada en privacidad y costo

🚀 Tarea para esta Semana

- 1. Instalar Ollama + Python 3.12 en tu laptop Windows
- 2. Ejecutar make run-local y probar /credit/analyze en Swagger
- 3. Crear cuenta gratuita en portal.azure.com y explorar AI Foundry
- 4. Ejecutar make docker-up y verificar Grafana en :3000
- 5. Iniciar el repositorio del proyecto con README y Makefile